LinkedList

1. Insert at the Head of a Linked List

You are given the head of a singly linked list and an integer value `val`.

Your task is to **insert a new node with value `val` at the beginning of the linked list** (before the current head) and return the **updated head** of the list.

Examples

Example 1

```
Input:  0 -> 1 -> 2 , val = 5
Output: 5 -> 0 -> 1 -> 2
```

Example 2

```
Input:  12 -> 5 -> 8 -> 7 , val = 4
Output: 4 -> 12 -> 5 -> 8 -> 7
```

Key Idea

In a linked list, the **head is just a pointer** to the first node.

To insert a node **before the head**, we:

1. Create a new node
2. Point its `next` to the current head
3. Make this new node the new head

No traversal is required.

Algorithm (Step-by-Step)

1. Create a new node with data = `val`
2. Set `newNode->next` to the current `head`
3. Return `newNode` as the updated head

Dry Run

Initial List

`head → 2 → 3 → NULL`

Insert value 1 at head

Step 1: Create new node

`newNode = 1`

Step 2: Point new node to old head

`1 → 2 → 3 → NULL`

Step 3: Update head

`head = newNode`

Final List:

`1 → 2 → 3 → NULL`

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

// Node class to represent each node in the linked list
class Node {
public:
    // Data stored in the node
    int data;
```

```
    // Pointer to the next node
    Node* next;

    // Constructor with data and next pointer
    Node(int data1, Node* next1) {
        data = data1;
        next = next1;
    }

    // Constructor with only data
    Node(int data1) {
        data = data1;
        next = nullptr;
    }
};

// Solution class to handle linked list operations
class Solution {
public:
    // Function to insert a new node at the head
    Node* insertAtHead(Node* head, int newData) {
        // Create a new node whose next points to current head
        Node* newNode = new Node(newData, head);
        // Return the new node as the head
        return newNode;
    }

    // Function to print the linked list
    void printList(Node* head) {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};
```

```
int main() {
    Solution sol;

    // Creating a sample linked list: 2 -> 3
    Node* head = new Node(2);
    head->next = new Node(3);

    cout << "Original List: ";
    sol.printList(head);

    // Inserting new node at head
    head = sol.insertAtHead(head, 1);

    cout << "After Insertion at Head: ";
    sol.printList(head);

    return 0;
}
```

Edge Case: Empty List

If the list is empty (`head == NULL`):

- The new node's `next` will be `NULL`
- The new node itself becomes the head

Works correctly without any special handling.

Complexity Analysis

Type	Complexity
Time Complexity	O(1)
Space Complexity	O(1)

Only one node is created and no traversal is done.

---

2. Delete Last Node of a Linked List

You are given the head of a **singly linked list**.

Your task is to **delete the last node (tail)** of the linked list and return the **updated head** of the list.

Examples

Example 1

```
Input:  0 → 1 → 2

Output: 0 → 1
```

Explanation:
The last node is 2, so it is removed.

Example 2

```
Input:  12 → 5 → 8 → 7

Output: 12 → 5 → 8
```

Explanation:
The last node is 7, so it is removed.

Key Idea

In a **singly linked list**, each node only knows about the **next node**.

To delete the last node:

- We must reach the **second last node**
- Set its `next` pointer to `NULL`
- Delete the old last node

Important Edge Cases

1️⃣ **Empty list (`head == NULL`)**
→ Nothing to delete, return `NULL`

2️⃣ **Only one node in the list**
→ Delete that node and return `NULL`

Algorithm (Step-by-Step)

1. If the list is empty or has only one node:
    - Delete the node (if any)
    - Return `NULL`
2. Otherwise:
    - Traverse the list until you reach the **second last node**
    - Delete the last node
    - Set `secondLast->next = NULL`
    - Return the head

Dry Run

Initial List

`head → 1 → 2 → 3 → NULL`

Traversal

We stop when:

```
curr->next->next == NULL
```

So `curr` points to:

```
1 → 2 → 3 → NULL
      ↑
    curr
```

Delete last node

```
delete curr->next;   // deletes node 3
curr->next = NULL;
```

Final List

`1 → 2 → NULL`

C++ Code

```
#include <bits/stdc++.h>

using namespace std;


// Definition for singly linked list

struct Node {

    int data;

    Node* next;

    Node(int val) {

        data = val;

        next = NULL;

    }

};


class Solution {

public:

    // Function to delete tail node of linked list

    Node* deleteTail(Node* head) {

        // If list is empty or has one node

        if (head == NULL || head->next == NULL) {

            delete head;

            return NULL;
```

```
        }

        // Traverse to the second last node
        Node* curr = head;
        while (curr->next->next != NULL) {
            curr = curr->next;
        }

        // Delete tail node
        delete curr->next;
        curr->next = NULL;

        // Return updated head
        return head;
    }
};

// Driver code
int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
```

```
    Solution obj;
    head = obj.deleteTail(head);

    // Print list after deletion
    Node* temp = head;
    while (temp) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    return 0;
}
```

Complexity Analysis

Type	Complexity
Time Complexity	O(N)
Space Complexity	O(1)

---

3. Find the Length of a Linked List

You are given the **head** of a singly linked list.

Your task is to **find and return the total number of nodes** present in the linked list.

Example 1

`Input: 0 → 1 → 2`

`Output: 3`

Explanation:
There are three nodes in the list, so the length is 3.

Example 2

`Input: 12 → 5 → 8 → 7`

`Output: 4`

Explanation:
There are four nodes in the list, so the length is 4.

Key Idea

A linked list is made up of nodes connected using `next` pointers.

To find its length:

- Start from the **head**
- Move node by node using `next`
- Count how many nodes you visit until you reach `NULL`

Algorithm (Step-by-Step)

1. Initialize a counter `count = 0`
2. Create a temporary pointer `temp` and point it to `head`
3. Traverse the list while `temp` is not `NULL`
   - Increment `count`
   - Move `temp` to `temp->next`
4. When traversal ends, return `count`

Dry Run

Linked List

`head → 10 → 20 → 30 → NULL`

Traversal

temp points to	coun
10	1
20	2
30	3
NULL	stop

Final Answer

```
Length = 3
```

C++ Code

```
#include <bits/stdc++.h>

using namespace std;


// Node class to represent each element in the linked list

class Node {

public:

    int data;

    Node* next;


    // Constructor to initialize data and next pointer

    Node(int data1) {

        data = data1;

        next = nullptr;

    }

};
```

```
// Solution class containing the method to find length
class Solution {
public:
    // Function to find the length of the linked list
    int lengthOfLinkedList(Node* head) {
        // Initialize counter to 0
        int count = 0;

        // Initialize a temporary pointer to head
        Node* temp = head;

        // Traverse the linked list
        while (temp != nullptr) {
            // Increment count for each node
            count++;

            // Move to the next node
            temp = temp->next;
        }

        // Return the total count
        return count;
    }
```

```
};


int main() {
    // Creating a sample linked list
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);

    // Create Solution object
    Solution obj;

    // Find and print the length of linked list
    cout << "Length of Linked List: "
         << obj.lengthOfLinkedList(head) << endl;

    return 0;
}
```

Edge Cases

1️⃣ Empty list

```
head = NULL
```

→ Loop does not run
→ `count = 0`
→ Correct answer

2️⃣ **Single node**

```
head → X → NULL
```

→ Loop runs once
→ `count = 1`

Complexity Analysis

Type	Complexity
Time Complexity	O(N)
Space Complexity	O(1)

4. Search an Element in a Linked List

You are given the head of a singly linked list and an integer value.
Check whether the given value exists in the linked list.

Idea:
Traverse the linked list from the head node.
At each node, compare its data with the given value.
If a match is found, return true.
If the traversal ends without a match, return false.

```cpp
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = nullptr;
    }
};

class Solution {
public:
    bool searchValue(Node* head, int val) {
        Node* temp = head;
        while (temp != nullptr) {
            if (temp->data == val)
                return true;
            temp = temp->next;
        }
        return false;
    }
};

int main() {
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);

    Solution sol;
    int val = 20;

    cout << (sol.searchValue(head, val) ? "True" : "False");
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

5. Introduction to Doubly Linked List

Before a doubly linked list, recall a **singly linked list**.
In a singly linked list, each node stores:

- data
- a pointer to the next node

Traversal is possible only in the forward direction. Moving backward is not possible because there is no reference to the previous node.

Singly Linked List Node Structure

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int data1, Node* next1) {
        data = data1;
        next = next1;
    }

    Node(int data1) {
        data = data1;
        next = nullptr;
    }
};
```

```
int main() {
    vector<int> arr = {2, 5, 8, 7};

    Node* head = new Node(arr[0]);

    cout << head << endl;
    cout << head->data << endl;

    return 0;
}
```

Doubly Linked List

A doubly linked list improves upon a singly linked list by allowing **two-way traversal**.
Each node stores:

- data
- pointer to the next node
- pointer to the previous node

Because of the `prev` pointer, traversal is possible both forward and backward.

Doubly Linked List Node Structure

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int data1, Node* next1, Node* prev1) {
```

```
        data = data1;
        next = next1;
        prev = prev1;
    }

    Node(int data1) {
        data = data1;
        next = nullptr;
        prev = nullptr;
    }
};

int main() {
    vector<int> arr = {2, 5, 8, 7};

    Node* head = new Node(arr[0]);

    cout << head << endl;
    cout << head->data << endl;

    return 0;
}
```

Key Difference:
Singly Linked List → only `next` pointer
Doubly Linked List → both `next` and `prev` pointers

6. Insert at End of Doubly Linked List

You are given the head of a doubly linked list and a value k.
Insert a new node with value k at the end of the list and return the head.

Idea:

If the list is empty, the new node becomes the head.

Otherwise, traverse to the last node (tail).

Link the new node after the tail and update both `next` and `prev` pointers.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int val) {
        data = val;
        next = nullptr;
        prev = nullptr;
    }
};

Node* insertAtEnd(Node* head, int k) {
    Node* newNode = new Node(k);

    if (head == nullptr)
        return newNode;

    Node* tail = head;
    while (tail->next != nullptr) {
        tail = tail->next;
    }

    tail->next = newNode;
    newNode->prev = tail;

    return head;
}

void printList(Node* head) {
```

```
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->prev = head;
    head->next->next = new Node(3);
    head->next->next->prev = head->next;

    head = insertAtEnd(head, 6);
    printList(head);

    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

7. Delete Last Node of a Doubly Linked List

You are given the head of a doubly linked list.
Delete the last node (tail) of the list and return the updated head.

Idea:
If the list is empty, nothing to delete.
If the list has only one node, delete it and return `NULL`.
Otherwise, traverse to the last node using `next`.

Use the `prev` pointer to reach the second last node.
Update the second last node's `next` to `NULL` and delete the tail.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* prev;
    Node* next;

    Node(int val) {
        data = val;
        prev = nullptr;
        next = nullptr;
    }
};

Node* deleteLastNode(Node* head) {
    if (head == nullptr)
        return nullptr;

    if (head->next == nullptr) {
        delete head;
        return nullptr;
    }

    Node* tail = head;
    while (tail->next != nullptr) {
        tail = tail->next;
    }

    Node* secondLast = tail->prev;
    secondLast->next = nullptr;
    delete tail;

    return head;
}
```

```
void printList(Node* head) {
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(3);
    head->next->prev = head;
    head->next->next = new Node(4);
    head->next->next->prev = head->next;
    head->next->next->next = new Node(1);
    head->next->next->next->prev = head->next->next;

    head = deleteLastNode(head);
    printList(head);

    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

8. Reverse a Doubly Linked List

You are given the head of a doubly linked list. Reverse the list and return the new head.

Brute Force Approach

Idea:
Store all node values in a stack.
Traverse the list again and replace node values using the stack (LIFO gives reverse order).

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node* prev;
    Node(int val) {
        data = val;
        next = nullptr;
        prev = nullptr;
    }
};

Node* reverseDLL(Node* head) {
    if (head == nullptr || head->next == nullptr)
        return head;

    stack<int> st;
    Node* temp = head;

    while (temp != nullptr) {
        st.push(temp->data);
        temp = temp->next;
    }
```

```
    temp = head;
    while (temp != nullptr) {
        temp->data = st.top();
        st.pop();
        temp = temp->next;
    }

    return head;
}

void printList(Node* head) {
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->prev = head;
    head->next->next = new Node(3);
    head->next->next->prev = head->next;

    head = reverseDLL(head);
    printList(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(N)

Optimal Approach

Idea:
Reverse the list by swapping `next` and `prev` pointers for every node.

While traversing, keep updating the head to the current node.
At the end, the last processed node becomes the new head.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node* prev;
    Node(int val) {
        data = val;
        next = nullptr;
        prev = nullptr; }
};

Node* reverseDLL(Node* head) {
    if (head == nullptr || head->next == nullptr)
        return head;

    Node* curr = head;

    while (curr != nullptr) {
        Node* temp = curr->next;
        curr->next = curr->prev;
        curr->prev = temp;
        head = curr;
        curr = temp;
    }

    return head;
}

void printList(Node* head) {
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }
```

```
}

int main() {
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->prev = head;
    head->next->next = new Node(30);
    head->next->next->prev = head->next;

    head = reverseDLL(head);
    printList(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

9. Find Middle Element in a Linked List

You are given the head of a linked list.
If the list has **odd nodes**, return the middle node.
If the list has **even nodes**, return the **second middle node**.

Brute Force Approach

Idea:
First count total nodes.
Then move to `(count / 2)` steps from head (0-indexed), which gives the middle.

```
#include <bits/stdc++.h>
```

```
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) {
        data = val;
        next = nullptr;
    }
};

Node* findMiddle(Node* head) {
    if (head == nullptr) return nullptr;

    int count = 0;
    Node* temp = head;

    while (temp != nullptr) {
        count++;
        temp = temp->next;
    }
```

```
    int mid = count / 2;
    temp = head;

    while (mid--) {
        temp = temp->next;
    }

    return temp;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    Node* ans = findMiddle(head);
    cout << ans->data;
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

Optimal Approach (Tortoise & Hare)

Idea:
Use two pointers:
`slow` moves 1 step, `fast` moves 2 steps.
When `fast` reaches the end, `slow` is at the middle.
For even length, this automatically gives the **second middle**.

```
#include <bits/stdc++.h>

using namespace std;


class Node {

public:

    int data;

    Node* next;

    Node(int val) {

        data = val;

        next = nullptr;

    }

};


Node* findMiddle(Node* head) {

    if (head == nullptr) return nullptr;
```

```
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);
    head->next->next->next->next->next = new Node(6);

    Node* ans = findMiddle(head);
    cout << ans->data;
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

10. Reverse a Linked List

You are given the head of a **singly linked list**.
Reverse the linked list and return the new head.

Brute Force Approach (Using Stack)

Idea:
Store node values in a stack, then overwrite the list in reverse order.

```
#include <bits/stdc++.h>
using namespace std;

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) {
        val = x;
        next = NULL;
    }
};

ListNode* reverseList(ListNode* head) {
    stack<int> st;
    ListNode* temp = head;

    while (temp != NULL) {
```

```
        st.push(temp->val);
        temp = temp->next;
    }

    temp = head;
    while (temp != NULL) {
        temp->val = st.top();
        st.pop();
        temp = temp->next;
    }

    return head;
}

int main() {
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);

    head = reverseList(head);

    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
    }
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(N)

Optimal Approach (Iterative Pointer Reversal)

Idea:
Reverse links by changing `next` pointers using three pointers.

```cpp
#include <bits/stdc++.h>
using namespace std;

class ListNode {
public:
    int val;
    ListNode* next;
    ListNode(int x) {
        val = x;
        next = NULL;
    }
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = NULL;
    ListNode* curr = head;

    while (curr != NULL) {
        ListNode* nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }

    return prev;
}

int main() {
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);

    head = reverseList(head);

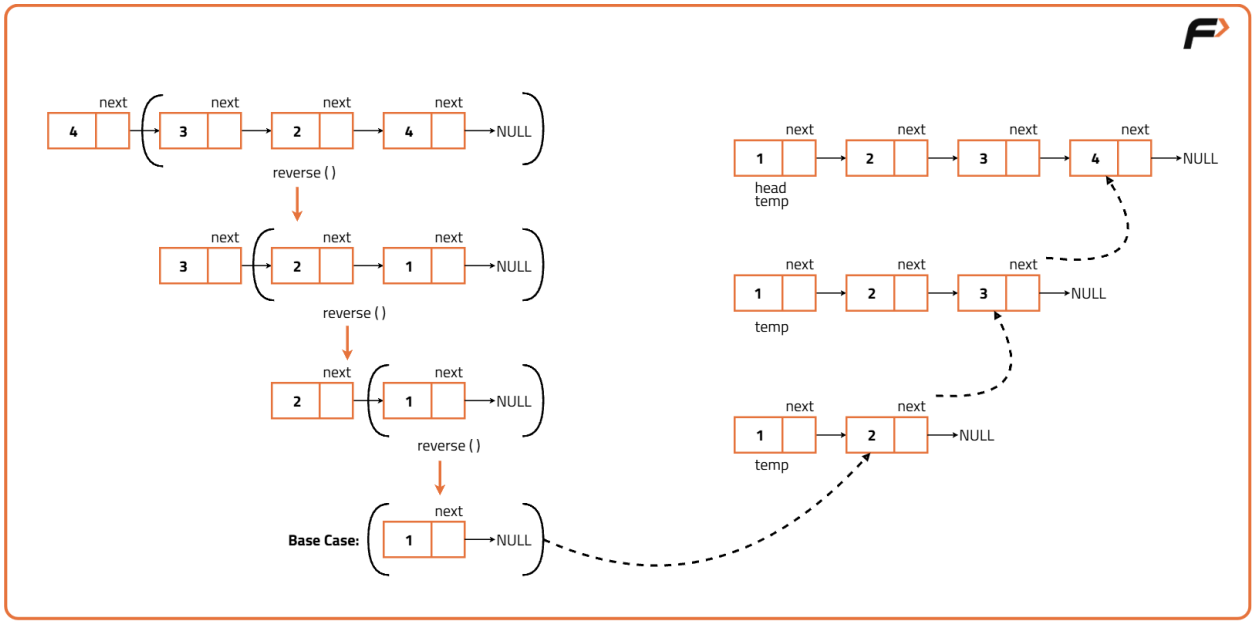
    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
```

```
    }
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

Recursive Approach

Idea:
Reverse rest of the list, then fix current node links while returning.




```
#include <bits/stdc++.h>
using namespace std;

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) {
        val = x;
```

```
        next = NULL;
    }
};

ListNode* reverseList(ListNode* head) {
    if (head == NULL || head->next == NULL)
        return head;

    ListNode* newHead = reverseList(head->next);

    head->next->next = head;
    head->next = NULL;

    return newHead;
}

int main() {
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);

    head = reverseList(head);

    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
    }
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(N) (recursion stack)

11. Detect a Cycle in a Linked List

You are given the head of a singly linked list.
Return **true** if the linked list contains a cycle, otherwise return **false**.

Brute Force Approach (Using Hashing)

Idea:
Keep track of visited nodes.
If a node is visited again, a cycle exists.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = NULL;
    }
};

bool detectCycle(Node* head) {
    unordered_set<Node*> visited;
    Node* temp = head;

    while (temp != NULL) {
        if (visited.count(temp))
            return true;
        visited.insert(temp);
        temp = temp->next;
    }
    return false;
}
```

```
int main() {
    Node* head = new Node(1);
    Node* n2 = new Node(2);
    Node* n3 = new Node(3);
    Node* n4 = new Node(4);

    head->next = n2;
    n2->next = n3;
    n3->next = n4;
    n4->next = n2;   // cycle

    cout << detectCycle(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(N)

Optimal Approach (Floyd's Cycle Detection)

Idea:
Use two pointers:

- `slow` moves one step
- `fast` moves two steps
  If they meet, a cycle exists.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
```

```
        next = NULL;
    }
};

bool detectCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return true;
    }
    return false;
}

int main() {
    Node* head = new Node(1);
    Node* n2 = new Node(2);
    Node* n3 = new Node(3);
    Node* n4 = new Node(4);

    head->next = n2;
    n2->next = n3;
    n3->next = n4;
    n4->next = n2;   // cycle

    cout << detectCycle(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

12. Starting Point of Loop in a Linked List

Given the head of a singly linked list, return the node where the cycle begins.
If there is no cycle, return NULL.

Brute Force Approach (Hashing)

Idea:
The first node that is visited again during traversal is the starting point of the loop.

```
#include <bits/stdc++.h>
using namespace std;

class ListNode {
public:
    int val;
    ListNode* next;
    ListNode(int x) {
        val = x;
        next = NULL;
    }
};

ListNode* detectCycle(ListNode* head) {
    unordered_set<ListNode*> visited;
    ListNode* temp = head;

    while (temp != NULL) {
        if (visited.count(temp))
            return temp;
        visited.insert(temp);
        temp = temp->next;
```

```
    }
    return NULL;
}

int main() {
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
    head->next->next->next->next = new ListNode(5);

    head->next->next->next->next->next = head->next->next; // loop at
3

    ListNode* ans = detectCycle(head);
    if (ans) cout << ans->val;
    else cout << "NULL";
    return 0;
}
```

Time Complexity: O(N)
Space Complexity: O(N)

Optimal Approach (Floyd's Cycle Detection)

Idea:
First detect the cycle using slow and fast pointers.
After they meet, reset one pointer to head.
Move both one step at a time — the meeting point is the start of the loop.

```
#include <bits/stdc++.h>
using namespace std;

class ListNode {
public:
    int val;
    ListNode* next;
    ListNode(int x) {
```

```
        val = x;
        next = NULL;
    }
};

ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            slow = head;
            while (slow != fast) {
                slow = slow->next;
                fast = fast->next;
            }
            return slow;
        }
    }
    return NULL;
}

int main() {
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
    head->next->next->next->next = new ListNode(5);

    head->next->next->next->next->next = head->next->next; // loop at
3

    ListNode* ans = detectCycle(head);
    if (ans) cout << ans->val;
    else cout << "NULL";
```

```
    return 0;
}
```

Time Complexity: O(N)
Space Complexity: O(1)

Calculation:

x = head se loop start tak distance

c = loop ka length

y = loop start se meeting point tak distance

Step 1: Slow aur Fast ne kitna distance chala?

Slow pointer

- Head se chala
- Loop me entry ki
- Meeting point tak gaya

```
Slow distance = x + y
```

Numbers daalo:

```
Slow = 2 + 2 = 4
```

Fast pointer

- Slow se **2× speed**
- Loop ke andar **extra rounds** laga sakta hai

Isliye:

```
Fast distance = x + y + k*c
```

Yahan:

- k = loop ke extra rounds

Step 2: Floyd ka rule use karo

Fast = 2 × Slow

```
x + y + k*c = 2(x + y)
```

Step 3: Equation solve karo

```
x + y + k*c = 2x + 2y
```

Left se `x + y` minus karo:

```
k*c = x + y
```

Step 4: x ke liye likho

```
x = k*c - y
```

Step 5: Numbers daal ke check karo

`x = 1*4 - 2`
`x = 2`

✔️ Match karta hai (head → loop start = 2)

Step 6: Iska matlab kya nikla?

`x = k*c - y`

Matlab:

- Head se loop start tak distance = **x**
- Meeting point se loop start tak distance:
    - Loop me aage jaate hue = `c - y`
    - Aur agar extra rounds chahiye → `k*c - y`

👉 Dono distance **same** hain

13. Length of Loop in Linked List

Given the head of a singly linked list, return the **length of the loop** if a cycle exists.
If there is **no loop**, return **0**.

Brute Force Approach (Hashing + Timer)

Idea:
Store each visited node with the step number at which it was visited.
When a node is visited again, the difference in steps gives the loop length.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = NULL;
    }
};

int lengthOfLoop(Node* head) {
    unordered_map<Node*, int> visited;
    int timer = 0;
    Node* temp = head;

    while (temp != NULL) {
        if (visited.count(temp)) {
            return timer - visited[temp];
        }
        visited[temp] = timer;
        timer++;
        temp = temp->next;
    }
    return 0;
}

int main() {
    Node* head = new Node(1);
    Node* n2 = new Node(2);
    Node* n3 = new Node(3);
```

```
    Node* n4 = new Node(4);
    Node* n5 = new Node(5);

    head->next = n2;
    n2->next = n3;
    n3->next = n4;
    n4->next = n5;
    n5->next = n3;   // loop of length 3

    cout << lengthOfLoop(head);
    return 0;
}
```

Time Complexity: O(N)
Space Complexity: O(N)

Optimal Approach (Floyd's Cycle Detection)

Idea:
First detect the loop using slow and fast pointers.
Once they meet, move one pointer around the loop until it comes back to the same node, counting steps.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = NULL;
    }
};

int lengthOfLoop(Node* head) {
    Node* slow = head;
```

```
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            int count = 1;
            Node* temp = slow->next;
            while (temp != slow) {
                count++;
                temp = temp->next;
            }
            return count;
        }
    }
    return 0;
}

int main() {
    Node* head = new Node(1);
    Node* n2 = new Node(2);
    Node* n3 = new Node(3);
    Node* n4 = new Node(4);
    Node* n5 = new Node(5);

    head->next = n2;
    n2->next = n3;
    n3->next = n4;
    n4->next = n5;
    n5->next = n3;   // loop of length 3

    cout << lengthOfLoop(head);
    return 0;
}
```

Time Complexity: O(N)
Space Complexity: O(1)

14. Check if the given Linked List is Palindrome

You are given the head of a singly linked list where each node stores a digit.
Check whether the sequence of digits forms a palindrome.

Brute Force Approach (Using Stack)

Idea:
Store all node values in a stack.
Traverse the list again and compare each node with the top of the stack.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = NULL;
    }
};

bool isPalindrome(Node* head) {
    stack<int> st;
    Node* temp = head;

    while (temp != NULL) {
        st.push(temp->data);
        temp = temp->next;
    }

    temp = head;
```

```
    while (temp != NULL) {
        if (temp->data != st.top())
            return false;
        st.pop();
        temp = temp->next;
    }
    return true;
}

int main() {
    Node* head = new Node(3);
    head->next = new Node(7);
    head->next->next = new Node(5);
    head->next->next->next = new Node(7);
    head->next->next->next->next = new Node(3);

    cout << (isPalindrome(head) ? "True" : "False");
    return 0;
}
```

Time Complexity: O(N)
Space Complexity: O(N)

Optimal Approach (Reverse Second Half)

Idea:
Find the middle using slow and fast pointers.
Reverse the second half of the list.
Compare first half and second half node by node.
Restore the list by reversing back.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
```

```
    Node(int data1) {
        data = data1;
        next = NULL;
    }
};

Node* reverseList(Node* head) {
    Node* prev = NULL;
    while (head != NULL) {
        Node* nextNode = head->next;
        head->next = prev;
        prev = head;
        head = nextNode;
    }
    return prev;
}

bool isPalindrome(Node* head) {
    if (head == NULL || head->next == NULL) return true;

    Node* slow = head;
    Node* fast = head;

    while (fast->next != NULL && fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    Node* second = reverseList(slow->next);
    Node* first = head;
    Node* temp = second;

    while (temp != NULL) {
        if (first->data != temp->data) {
            reverseList(second);
            return false;
        }
        first = first->next;
```

```
        temp = temp->next;
    }

    reverseList(second);
    return true;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(1);
    head->next->next = new Node(2);
    head->next->next->next = new Node(1);

    cout << (isPalindrome(head) ? "True" : "False");
    return 0;
}
```

Time Complexity: O(N)
Space Complexity: O(1)

15. Segregate even and odd nodes in Linked List

You are given the head of a singly linked list.
Reorder the list so that **all even-valued nodes come first**, followed by **all odd-valued nodes**.
The **relative order inside even and odd groups must remain the same**.

Idea

We do **not** change node values.
We only **rearrange links**.

Create two separate lists while traversing once:

- one list for even-valued nodes
- one list for odd-valued nodes

At the end, connect:
`even list → odd list`

This keeps order intact and avoids extra traversals.

Code (single pass, pointer-based)

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int x) {
        data = x;
        next = NULL;
    }
};

Node* segregateEvenOdd(Node* head) {
    if (head == NULL) return head;

    Node* evenDummy = new Node(-1);
    Node* oddDummy = new Node(-1);
    Node* evenTail = evenDummy;
    Node* oddTail = oddDummy;

    Node* curr = head;
    while (curr != NULL) {
        if (curr->data % 2 == 0) {
            evenTail->next = curr;
            evenTail = evenTail->next;
```

```
        } else {
            oddTail->next = curr;
            oddTail = oddTail->next;
        }
        curr = curr->next;
    }

    evenTail->next = oddDummy->next;
    oddTail->next = NULL;

    return evenDummy->next;
}

void printList(Node* head) {
    while (head != NULL) {
        cout << head->data << "->";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);
    head->next->next->next->next->next = new Node(6);

    printList(head);
    head = segregateEvenOdd(head);
    printList(head);

    return 0;
}
```

Time Complexity: **O(N)**
Space Complexity: **O(1)** (only pointer manipulation, no extra data structures)

16. Remove N-th node from the end of a Linked List

You are given the head of a singly linked list and an integer **N**.
Delete the **N-th node from the end** of the list and return the updated head.

Brute Force Approach

Idea:
Find the length of the linked list first.
The N-th node from the end is the `(length - N + 1)`-th node from the start.

Algorithm:

1. Traverse the list and count total nodes L.
2. If `N == L`, delete the head node.
3. Traverse again to the `(L - N)`-th node.
4. Adjust pointers to remove the next node.

```
#include <bits/stdc++.h>

using namespace std;


class Node {
```

```
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = nullptr;
    }
};

Node* removeNthFromEnd(Node* head, int N) {
    if (!head) return nullptr;

    int len = 0;
    Node* temp = head;
    while (temp) {
        len++;
        temp = temp->next;
    }

    if (N == len) {
        Node* newHead = head->next;
```

```
        delete head;
        return newHead;
    }

    temp = head;
    for (int i = 1; i < len - N; i++) {
        temp = temp->next;
    }

    Node* del = temp->next;
    temp->next = del->next;
    delete del;

    return head;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
```

```
    head->next->next->next->next = new Node(5);


    head = removeNthFromEnd(head, 3);


    while (head) {
        cout << head->data << " ";
        head = head->next;
    }
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

Idea (two-pointer method)

Instead of counting length, use **two pointers**.

Keep a gap of **N nodes** between `fast` and `slow`.

- Move `fast` ahead by N+1 steps
- Move both `fast` and `slow` together
- When `fast` reaches `NULL`, `slow` is just **before** the node to delete
- Bypass that node

Using a **dummy node** simplifies edge cases (like deleting the head).

Code (optimal, single pass)

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int x) {
        data = x;
        next = NULL;
    }
};

Node* removeNthFromEnd(Node* head, int N) {
    Node* dummy = new Node(0);
    dummy->next = head;

    Node* slow = dummy;
    Node* fast = dummy;

    for (int i = 0; i <= N; i++) {
        fast = fast->next;
    }

    while (fast != NULL) {
        slow = slow->next;
        fast = fast->next;
    }

    Node* delNode = slow->next;
    slow->next = slow->next->next;
    delete delNode;

    return dummy->next;
```

```
}

void printList(Node* head) {
    while (head != NULL) {
        cout << head->data << "->";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    int N = 3;

    printList(head);
    head = removeNthFromEnd(head, N);
    printList(head);

    return 0;
}
```

Why this works

When `fast` reaches the end, it has moved **N nodes more** than `slow`.
So `slow` is exactly **one node before** the target node.

Complexity

Time Complexity: **O(N)**
Space Complexity: **O(1)**

17. Delete the Middle Node of the Linked List

You are given the head of a singly linked list.
Delete the **middle node** and return the updated head.
If the list has an **even number of nodes**, delete the **second middle** node.

Brute Force Approach

Idea:
Find the length of the list first.
Middle index = n / 2 (0-based logic → this deletes second middle in even case).
Traverse again and delete that node.

Steps:

1. Count total nodes n.
2. If n == 1, return NULL.
3. Traverse to node just **before** middle.
4. Adjust pointers and delete the middle node.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = nullptr;
    }
};
```

```
Node* deleteMiddle(Node* head) {
    if (head == nullptr || head->next == nullptr) {
        delete head;
        return nullptr;
    }

    int n = 0;
    Node* temp = head;
    while (temp) {
        n++;
        temp = temp->next;
    }

    int mid = n / 2;
    temp = head;

    for (int i = 1; i < mid; i++) {
        temp = temp->next;
    }

    Node* del = temp->next;
    temp->next = del->next;
    delete del;

    return head;
}

void printLL(Node* head) {
    while (head) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    Node* head = new Node(1);
```

```
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    head = deleteMiddle(head);
    printLL(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

Optimal Approach (Slow & Fast Pointer)

Idea:
Use two pointers.
`fast` moves two steps, `slow` moves one step.
We stop `slow` **one node before** the middle, then delete `slow->next`.

This automatically deletes:

- exact middle for odd length
- **second middle** for even length

Steps:

1. Handle 0 or 1 node case.
2. Initialize `slow = head`, `fast = head->next->next`.
3. Move `slow` by 1 and `fast` by 2.
4. Delete `slow->next`.

```cpp
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = nullptr;
    }
};

Node* deleteMiddle(Node* head) {
    if (head == nullptr || head->next == nullptr) {
        delete head;
        return nullptr;
    }

    Node* slow = head;
    Node* fast = head->next->next;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    Node* del = slow->next;
    slow->next = del->next;
    delete del;

    return head;
}

void printLL(Node* head) {
    while (head) {
        cout << head->data << " ";
        head = head->next;
```

```
    }
    cout << endl;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);

    head = deleteMiddle(head);
    printLL(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

18. Sort a Linked List

You are given the head of a singly linked list.
Sort the linked list in **ascending order** and return the head of the sorted list.

Brute Force Approach

Idea:
Store all node values in an array, sort the array, then overwrite the linked list with sorted values.

Steps:

1. Traverse the linked list and store values in an array.
2. Sort the array.

3. Traverse the linked list again and replace node values from the sorted array.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = nullptr;
    }
};

Node* sortLL(Node* head) {
    vector<int> arr;
    Node* temp = head;

    while (temp) {
        arr.push_back(temp->data);
        temp = temp->next;
    }

    sort(arr.begin(), arr.end());

    temp = head;
    for (int i = 0; i < arr.size(); i++) {
        temp->data = arr[i];
        temp = temp->next;
    }

    return head;
}

void printLL(Node* head) {
    while (head) {
        cout << head->data << " ";
```

```
        head = head->next;
    }
    cout << endl;
}

int main() {
    Node* head = new Node(3);
    head->next = new Node(4);
    head->next->next = new Node(2);
    head->next->next->next = new Node(1);
    head->next->next->next->next = new Node(5);

    head = sortLL(head);
    printLL(head);
    return 0;
}
```

Complexity:
Time: `O(N log N)`
Space: `O(N)`

Optimal Approach (Merge Sort on Linked List)

Idea:
Linked lists are best sorted using **Merge Sort** because it does not require random access.

Steps:

1. If list has 0 or 1 node, it is already sorted.
2. Find the middle using slow & fast pointers.
3. Split the list into two halves.
4. Recursively sort both halves.
5. Merge the two sorted halves.

```cpp
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int data1) {
        data = data1;
        next = nullptr;
    }
};

Node* merge(Node* l1, Node* l2) {
    Node dummy(-1);
    Node* temp = &dummy;

    while (l1 && l2) {
        if (l1->data <= l2->data) {
            temp->next = l1;
            l1 = l1->next;
        } else {
            temp->next = l2;
            l2 = l2->next;
        }
        temp = temp->next;
    }

    temp->next = (l1 ? l1 : l2);
    return dummy.next;
}

Node* findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head->next;

    while (fast && fast->next) {
        slow = slow->next;
```

```
        fast = fast->next->next;
    }
    return slow;
}

Node* sortLL(Node* head) {
    if (!head || !head->next) return head;

    Node* mid = findMiddle(head);
    Node* right = mid->next;
    mid->next = nullptr;

    Node* left = sortLL(head);
    right = sortLL(right);

    return merge(left, right);
}

void printLL(Node* head) {
    while (head) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    Node* head = new Node(40);
    head->next = new Node(20);
    head->next->next = new Node(60);
    head->next->next->next = new Node(10);
    head->next->next->next->next = new Node(50);
    head->next->next->next->next->next = new Node(30);

    head = sortLL(head);
    printLL(head);
    return 0;
}
```

Complexity:
Time: `O(N log N)`
Space: `O(1)` (excluding recursion stack)

19. Sort a Linked List of 0's, 1's and 2's (by changing links)

You are given a singly linked list containing only `0`, 1, and 2.
You must sort the list **by rearranging links**, not by modifying node data.

Brute Force Approach

Idea:
Count the number of `0`s, 1s, and 2s, then rebuild the linked list in sorted order by **creating new links**.

Steps:

1. Traverse the linked list and count occurrences of `0`, 1, and 2.
2. Traverse the list again and overwrite links in order: first `0`s, then 1s, then 2s.
3. Do not create new data values, only reuse existing nodes.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int d) {
```

```
        data = d;
        next = nullptr;
    }
};

Node* sortList(Node* head) {
    int cnt0 = 0, cnt1 = 0, cnt2 = 0;
    Node* temp = head;

    while (temp) {
        if (temp->data == 0) cnt0++;
        else if (temp->data == 1) cnt1++;
        else cnt2++;
        temp = temp->next;
    }

    temp = head;
    while (cnt0--) {
        temp->data = 0;
        temp = temp->next;
    }
    while (cnt1--) {
        temp->data = 1;
        temp = temp->next;
    }
    while (cnt2--) {
        temp->data = 2;
        temp = temp->next;
    }
    return head;
}

void printLL(Node* head) {
    while (head) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
```

```
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(0);
    head->next->next->next = new Node(1);
    head->next->next->next->next = new Node(0);
    head->next->next->next->next->next = new Node(2);

    head = sortList(head);
    printLL(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

Optimal Approach (Link Rearrangement)

Idea:
Create **three separate linked lists** for 0, 1, and 2 using dummy nodes.
Traverse once, attach each node to its respective list, then merge all three lists.

Steps:

1. Create dummy heads and tails for lists of 0, 1, and 2.
2. Traverse original list and attach nodes to corresponding list.
3. Connect 0 list → 1 list → 2 list.
4. Update head to the start of sorted list.

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Node {
public:
    int data;
    Node* next;
    Node(int d) {
        data = d;
        next = nullptr;
    }
};

Node* sortList(Node* head) {
    if (!head || !head->next) return head;

    Node* zeroDummy = new Node(-1);
    Node* oneDummy = new Node(-1);
    Node* twoDummy = new Node(-1);

    Node* zero = zeroDummy;
    Node* one = oneDummy;
    Node* two = twoDummy;

    Node* curr = head;
    while (curr) {
        if (curr->data == 0) {
            zero->next = curr;
            zero = zero->next;
        } else if (curr->data == 1) {
            one->next = curr;
            one = one->next;
        } else {
            two->next = curr;
            two = two->next;
        }
        curr = curr->next;
    }

    zero->next = (oneDummy->next) ? oneDummy->next : twoDummy->next;
```

```
    one->next = twoDummy->next;
    two->next = nullptr;

    head = zeroDummy->next;

    delete zeroDummy;
    delete oneDummy;
    delete twoDummy;

    return head;
}

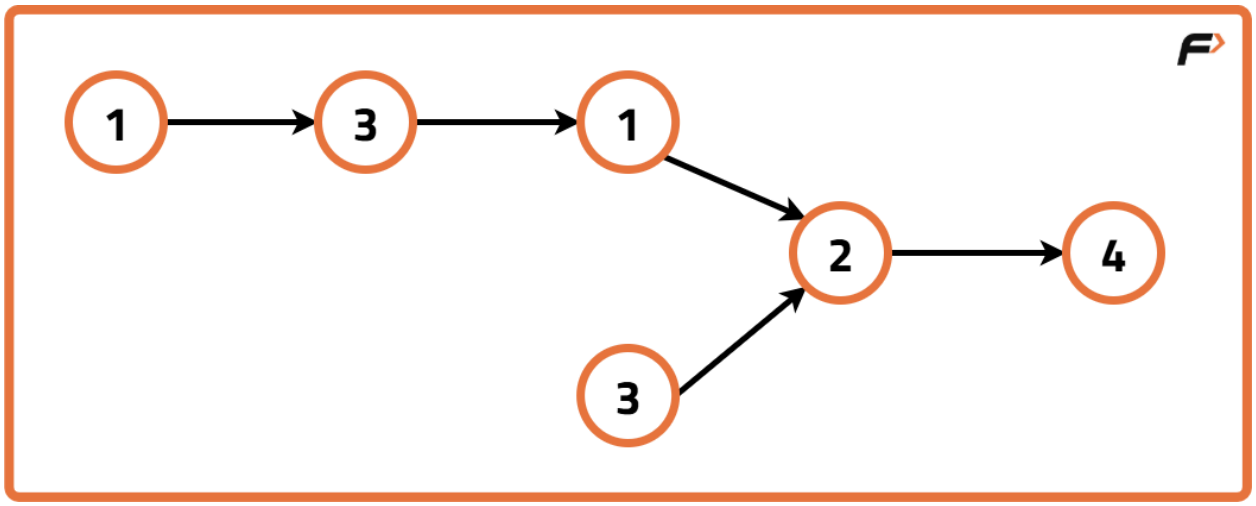
void printLL(Node* head) {
    while (head) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    Node* head = new Node(2);
    head->next = new Node(1);
    head->next->next = new Node(2);
    head->next->next->next = new Node(0);
    head->next->next->next->next = new Node(0);
    head->next->next->next->next->next = new Node(1);

    head = sortList(head);
    printLL(head);
    return 0;
}
```

Complexity:
Time: O(N)
Space: O(1)

Find intersection of Two Linked Lists




```
Example 1:Input:
List 1 = [1,3,1,2,4], List 2 = [3,2,4]
Output:
2
Explanation: Here, both lists intersecting nodes start from node 2.
```

Brute Force Approach

Idea:
Intersection means **same node address**, not same value.

Algorithm:

- For every node of list2, traverse list1.
- If `temp == head2`, intersection found.
- If traversal ends, return `NULL`.

```
node* intersectionPresent(node* head1, node* head2) {
    while (head2 != NULL) {
        node* temp = head1;
        while (temp != NULL) {
            if (temp == head2) return head2;
            temp = temp->next;
        }
```

```
        head2 = head2->next;
    }
    return NULL;
}
```

Complexity:

- Time: `O(m*n)`
- Space: `O(1)`

Better Approach (Hashing)

Idea:
Store node **addresses** of list1, then search while traversing list2.

Algorithm:

- Traverse list1, store each node in hash set.
- Traverse list2, if node exists in set → intersection.

```
node* intersectionPresent(node* head1, node* head2) {
    unordered_set<node*> st;
    while (head1 != NULL) {
        st.insert(head1);
        head1 = head1->next;
    }
    while (head2 != NULL) {
        if (st.find(head2) != st.end()) return head2;
        head2 = head2->next;
    }
    return NULL;
}
```

Complexity:

- Time: `O(n + m)`
- Space: `O(n)`

Optimal Approach 1 (Length Difference)

Idea:
Align both lists at same remaining length, then move together.

Algorithm:

- Find lengths of both lists.
- Move longer list by `|len1 - len2|`.
- Move both pointers together until they meet.

```
node* intersectionPresent(node* head1, node* head2) {
    int len1 = 0, len2 = 0;
    node* t1 = head1;
    node* t2 = head2;

    while (t1) { len1++; t1 = t1->next; }
    while (t2) { len2++; t2 = t2->next; }

    t1 = head1;
    t2 = head2;

    if (len1 > len2)
        for (int i = 0; i < len1 - len2; i++) t1 = t1->next;
    else
        for (int i = 0; i < len2 - len1; i++) t2 = t2->next;

    while (t1 && t2) {
        if (t1 == t2) return t1;
        t1 = t1->next;
        t2 = t2->next;
```

```
    }
    return NULL;
}
```

Complexity:

- Time: `O(n + m)`
- Space: `O(1)`

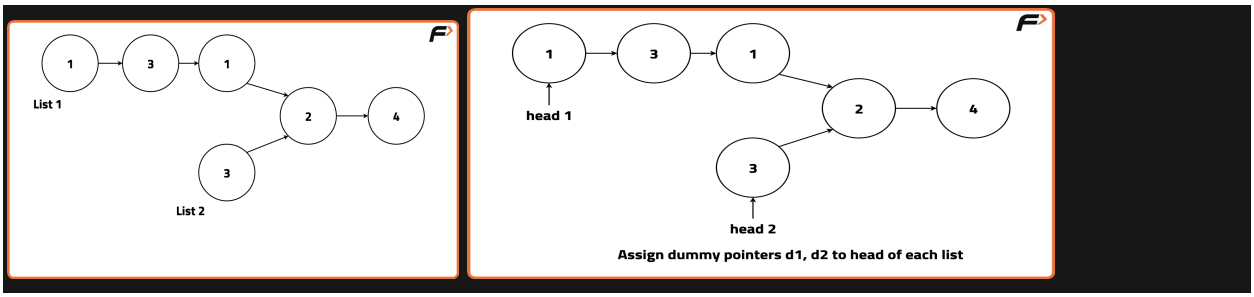
Optimal Approach 2 (Two Pointer Switching)

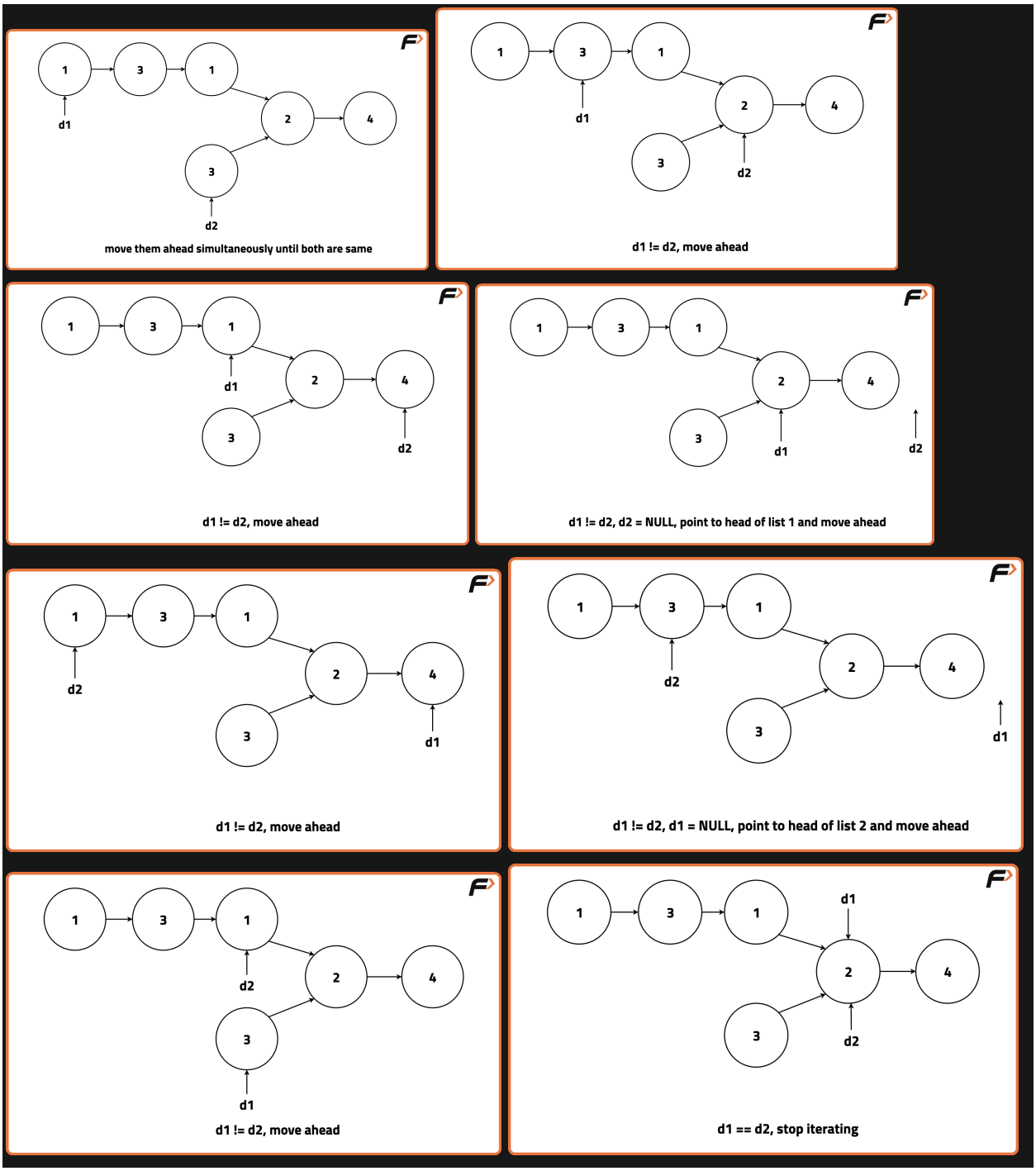
Idea:
Both pointers traverse equal total distance by switching lists.

Algorithm:

- Initialize `d1 = head1`, `d2 = head2`.
- Move both; when one hits NULL, redirect to other list.
- Meeting point is intersection.

Dry Run:





Dono pointers ko **same total distance** chalwa do.

List1 length = L1

List2 length = L2

Common tail length = C

d1 travels = L1 + L2

d2 travels = L2 + L1

```
#include<bits/stdc++.h>
using namespace std;

class node {
public:
    int num;
    node* next;
    node(int val) {
        num = val;
        next = NULL;
    }
};

// Utility function to insert node at the end of the linked list
void insertNode(node* &head, int val) {
    node* newNode = new node(val);

    if (head == NULL) {
        head = newNode;
        return;
    }

    node* temp = head;
    while (temp->next != NULL) temp = temp->next;

    temp->next = newNode;
}

// Utility function to check presence of intersection
node* intersectionPresent(node* head1, node* head2) {
    node* d1 = head1;
    node* d2 = head2;

    // Traverse both lists, when one reaches the end, redirect it to the head
of the other list
    while (d1 != d2) {
        d1 = d1 == NULL ? head2 : d1->next;
        d2 = d2 == NULL ? head1 : d2->next;
    }
```

```
    return d1;  // If they meet, return the intersection node, otherwise NULL
}

// Utility function to print linked list
void printList(node* head) {
    while (head->next != NULL) {
        cout << head->num << "->";
        head = head->next;
    }
    cout << head->num << endl;
}

int main() {
    node* head = NULL;
    insertNode(head, 1);
    insertNode(head, 3);
    insertNode(head, 1);
    insertNode(head, 2);
    insertNode(head, 4);
    node* head1 = head;
    head = head->next->next->next;  // Intersection point
    node* headSec = NULL;
    insertNode(headSec, 3);
    node* head2 = headSec;
    headSec->next = head;  // Creating intersection

    // Printing the lists
    cout << "List1: ";
    printList(head1);
    cout << "List2: ";
    printList(head2);

    // Checking if intersection is present
    node* answerNode = intersectionPresent(head1, head2);
    if (answerNode == NULL)
        cout << "No intersection\n";
    else
        cout << "The intersection point is " << answerNode->num << endl;

    return 0;
 }
```

Complexity:

- Time: `O(n + m)`
- Space: `O(1)`

20. Add 1 to a number represented by LL

Iterative approach

Algorithm

- Reverse the linked list.
- Add 1 starting from the head (least significant digit).
- Propagate carry if needed.
- If carry remains at the end, create a new node.
- Reverse the list again and return head.

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int value) {
        data = value;
```

```
        next = nullptr;
    }
};

class Solution {
public:
    Node* reverseList(Node* head) {
        Node* prev = nullptr;
        Node* curr = head;
        while (curr) {
            Node* nextNode = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nextNode;
        }
        return prev;
    }

    Node* addOne(Node* head) {
        head = reverseList(head);

        Node* curr = head;
        int carry = 1;

        while (curr && carry) {
            int sum = curr->data + carry;
            curr->data = sum % 10;
            carry = sum / 10;

            if (!curr->next && carry) {
                curr->next = new Node(carry);
                carry = 0;
            }
            curr = curr->next;
        }

        head = reverseList(head);
        return head;
```

```
    }
};

int main() {
    Node* head = new Node(4);
    head->next = new Node(5);
    head->next->next = new Node(6);

    Solution sol;
    head = sol.addOne(head);

    while (head) {
        cout << head->data;
        head = head->next;
    }
}
```

Complexity

- Time: $O(n)$
- Space: $O(1)$

Recursive approach

Algorithm

- Recursively go to the last node.
- Add 1 and propagate carry backwards.
- If carry remains after head, create a new node at front.

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Node {
public:
    int data;
    Node* next;
    Node(int value) {
        data = value;
        next = nullptr;
    }
};

class Solution {
public:
    int addOneUtil(Node* node) {
        if (!node) return 1;

        int carry = addOneUtil(node->next);
        int sum = node->data + carry;
        node->data = sum % 10;
        return sum / 10;
    }

    Node* addOne(Node* head) {
        int carry = addOneUtil(head);
        if (carry) {
            Node* newHead = new Node(carry);
            newHead->next = head;
            head = newHead;
        }
        return head;
    }
};

int main() {
    Node* head = new Node(9);
    head->next = new Node(9);
    head->next->next = new Node(9);

    Solution sol;
```

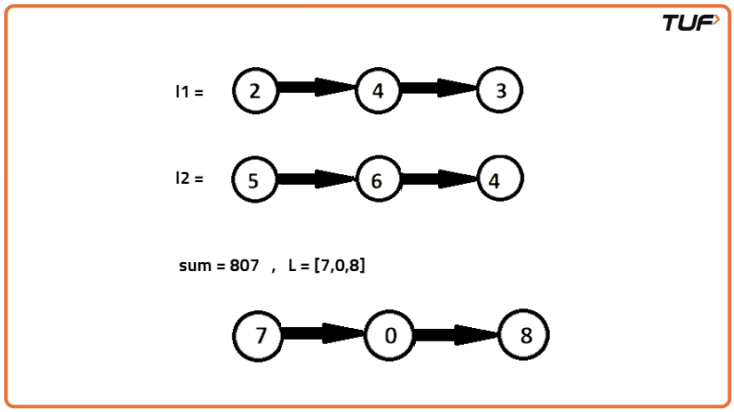
```
    head = sol.addOne(head);

    while (head) {
        cout << head->data;
        head = head->next;
    }
}
```

Complexity

- Time: $O(n)$
- Space: $O(n)$ (recursion stack)

21. Add two numbers represented as Linked Lists



Approach

Algorithm

- Create a dummy node to simplify result list creation.
- Use a temporary pointer starting at the dummy.
- Maintain a `carry` initialized to `0`.
- Traverse both linked lists while at least one list has nodes left **or** carry is non-zero.
- At each step:
    - Add current digits from both lists (if present) and the carry.
    - Update carry as `sum / 10`.
    - Create a new node with value `sum % 10` and attach it.
- Return `dummy->next` as the head of the result list.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Definition for singly-linked list.
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x = 0) {
        val = x;
        next = nullptr;
    }
};

class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* dummy = new ListNode();
        ListNode* temp = dummy;
        int carry = 0;

        while (l1 != nullptr || l2 != nullptr || carry) {
            int sum = carry;

            if (l1) {
                sum += l1->val;
                l1 = l1->next;
            }
            if (l2) {
                sum += l2->val;
                l2 = l2->next;
            }

            carry = sum / 10;
            temp->next = new ListNode(sum % 10);
            temp = temp->next;
        }
        return dummy->next;
```

```
    }
};

ListNode* createList(vector<int>& arr) {
    ListNode* head = new ListNode(arr[0]);
    ListNode* temp = head;
    for (int i = 1; i < arr.size(); i++) {
        temp->next = new ListNode(arr[i]);
        temp = temp->next;
    }
    return head;
}

void printList(ListNode* head) {
    while (head) {
        cout << head->val;
        if (head->next) cout << " -> ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    vector<int> num1 = {2, 4, 3};
    vector<int> num2 = {5, 6, 4};

    ListNode* l1 = createList(num1);
    ListNode* l2 = createList(num2);

    Solution sol;
    ListNode* result = sol.addTwoNumbers(l1, l2);

    printList(result); // 7 -> 0 -> 8
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(max(m, n))`
- **Space Complexity:** `O(max(m, n))`

22. Delete all occurrences of a given key in a doubly linked list

---

Intuition

We just need to **traverse the DLL once** and whenever we find a node whose `data == key`, we **delete that node and fix the links** (`prev` and `next`).
Since it's a doubly linked list, deletion can be done in **O(1)** time if we already have the node reference.

Approach

Algorithm

- If the list is empty, do nothing.
- Use a `current` pointer starting from head.
- Traverse the list:
    - If `current->data == key`:

        - Store `current->next` in a temporary pointer.
        - Delete `current` node using a helper function.
        - Move `current` to the saved next node.
    - Else:
        - Move `current` forward normally.
- Continue till the end of the list.

A helper function `deleteNode` is used to safely delete a node from a doubly linked list by updating adjacent pointers.

Code

```
class Solution {
public:
   Node* deleteAllOccurOfX(Node* head, int key) {
      if (head == NULL) return NULL;

      Node* curr = head;

      while (curr != NULL) {
         if (curr->data == key) {
            Node* nextNode = curr->next;

            // If node to delete is head
            if (curr == head) {
               head = head->next;
               if (head != NULL)
                  head->prev = NULL;
            } else {
               // Fix previous node
               curr->prev->next = curr->next;

               // Fix next node
               if (curr->next != NULL)
                  curr->next->prev = curr->prev;
            }
```

```
            delete curr;
            curr = nextNode;
        } else {
            curr = curr->next;
        }
    }

    return head;
  }
};
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each node is visited once, and deletion is `O(1)`.

- **Space Complexity:** `O(1)`
  No extra data structures are used.

23. Find pairs with given sum in doubly linked list

List **sorted** hai → Two Pointer best rahega.
DLL me advantage ye hai ki hum **prev** bhi use kar sakte hain.

- `left` = head
- `right` = last node
- Dono side se move karke pairs find karte hain

Algorithm

1. Agar list empty ho → empty answer return
2. `right` pointer ko last node tak le jao
3. `left = head`
4. Jab tak `left` aur `right` cross na kare:
    - `sum = left->data + right->data`
    - Agar `sum == target` → pair store, `left++`, `right--`
    - Agar `sum < target` → `left++`
    - Agar `sum > target` → `right--`
5. Answer return karo

Code (simple, clean, previous trend follow)

```
class Solution {
public:
    vector<pair<int,int>> findPairsWithGivenSum(Node* head, int
target) {
        vector<pair<int,int>> ans;
        if (head == NULL) return ans;
```

```
        Node* left = head;
        Node* right = head;

        // move right to last node
        while (right->next != NULL) {
            right = right->next;
        }

        // two pointer approach
        while (left != right && right->next != left) {
            int sum = left->data + right->data;

            if (sum == target) {
                ans.push_back({left->data, right->data});
                left = left->next;
                right = right->prev;
            }
            else if (sum < target) {
                left = left->next;
            }
            else {
                right = right->prev;
            }
        }

        return ans;
    }
};
```

Complexity

- **Time:** $O(N)$
- **Space:** $O(1)$ (answer vector ke alawa)

24. Remove duplicates from sorted DLL

Idea

List **sorted** hai → duplicates **adjacent** honge.
Har node ko uske `next` se compare karo.
Agar same value mile → duplicate node delete karo aur links adjust karo.

Algorithm

1. Agar list empty ho → return head
2. `curr` pointer ko head par rakho
3. Jab tak `curr` aur `curr->next` exist kare:
    - Agar `curr->data == curr->next->data`
        - duplicate node delete karo
        - `curr->next` ko aage wale node se jodo
    - warna `curr = curr->next`
4. Head return karo

Code (simple, previous trend follow)

```
class Solution {
public:
    Node* removeDuplicates(Node* head) {
        if (head == NULL) return head;
```

```
        Node* curr = head;

        while (curr != NULL && curr->next != NULL) {
            if (curr->data == curr->next->data) {
                Node* dup = curr->next;
                curr->next = dup->next;

                if (dup->next != NULL) {
                    dup->next->prev = curr;
                }

                delete dup;
            }
            else {
                curr = curr->next;
            }
        }

        return head;
    }
};
```

Complexity

- **Time:** O(N)
- **Space:** O(1)

25. Reverse Linked List in groups of Size K

Idea

Linked list ko **k size ke groups** me todte hain.
Har group me **sirf links reverse** karte hain (data change nahi).
Agar last group me **k se kam nodes** hain → usko as-it-is chhod dete hain.

Algorithm

1. Dummy node use karo taaki head change easily handle ho sake
2. `groupPrev` pointer rakho jo previous reversed group ke end ko point kare
3. Har iteration me:
    - `getKthNode` se check karo kya k nodes available hain
    - Agar nahi → break
    - Current group ko reverse karo using pointer reversal
    - Previous group ko current reversed group se connect karo
4. Last me `dummy->next` return karo

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
// Definition of singly-linked list node
class ListNode {
public:
    int val;
    ListNode* next;
    ListNode(int x) {
        val = x;
        next = NULL;
    }
};

class Solution {
public:
    // Function to reverse nodes in groups of k
    ListNode* reverseKGroup(ListNode* head, int k) {
        // Creating a dummy node to handle edge cases easily
        ListNode* dummy = new ListNode(0);
        dummy->next = head;

        // Pointer to keep track of the previous group's tail
        ListNode* groupPrev = dummy;

        while (true) {
            // Finding the k-th node from the groupPrev
            ListNode* kth = getKthNode(groupPrev, k);
            if (!kth) break;

            // Store the next group's head
            ListNode* groupNext = kth->next;

            // Break the chain to reverse current k-group cleanly
            ListNode* prev = groupNext;  //next group
            ListNode* curr = groupPrev->next;

            // Reversing k nodes
            for (int i = 0; i < k; i++) {
                ListNode* temp = curr->next;
                curr->next = prev;
                prev = curr;
                curr = temp;
            }

            // Connecting previous group to the reversed group
            ListNode* temp = groupPrev->next;
            groupPrev->next = kth;
            groupPrev = temp;
        }
```

```
        // Returning the new head
        return dummy->next;
    }

    // Helper function to find the k-th node from the current node
    ListNode* getKthNode(ListNode* curr, int k) {
        while (curr && k > 0) {
            curr = curr->next;
            k--;
        }
        return curr;
    }
};

// Driver code
int main() {
    Solution obj;

    // Creating the linked list: 1->2->3->4->5
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
    head->next->next->next->next = new ListNode(5);

    int k = 2;

    // Reversing in groups of k
    ListNode* result = obj.reverseKGroup(head, k);

    // Printing the reversed list
    while (result != NULL) {
        cout << result->val << " ";
        result = result->next;
    }
    cout << endl;

    return 0;
}
```

Complexity

- **Time Complexity:** $O(N)$

- **Space Complexity:** O(1)

26. Rotate a Linked List

Input : head -> 1 -> 2 -> 3 -> 4 -> 5, k = 2
Output : head -> 4 -> 5 -> 1 -> 2 -> 3
Explanation : List after 1 shift to right: head -> 5 -> 1 -> 2 -> 3 -> 4.
List after 2 shift to right: head -> 4 -> 5 -> 1 -> 2 -> 3.

Brute Force Approach

Idea

Right rotation ka matlab: **last node ko uthao aur head bana do**.
Ye process **k times** repeat karte hain.

Algorithm

1. Agar list empty ho, ek hi node ho, ya k = 0 ho → head return karo
2. k times repeat karo:
    - List ko traverse karke **last aur second-last node** tak jao
    - Last node ko detach karo
    - Usko head ke aage laga do
3. Final head return karo

Code (Brute Force)

```
class ListNode {
public:
```

```
    int val;
    ListNode* next;
    ListNode(int x) {
        val = x;
        next = NULL;
    }
};

class Solution {
public:
    ListNode* rotateRight(ListNode* head, int k) {
        if (!head || !head->next || k == 0) return head;

        while (k--) {
            ListNode* curr = head;
            ListNode* prev = NULL;

            while (curr->next) {
                prev = curr;
                curr = curr->next;
            }

            prev->next = NULL;
            curr->next = head;
            head = curr;
        }
        return head;
    }
};
```

Complexity

- **Time:** `O(k * N)`
- **Space:** `O(1)`

Optimal Approach

Har `length` rotations ke baad list wapas same ho jaati hai.
Isliye:

- `k = k % length`
- List ko **circular** bana do
- Naya tail aur naya head find karke circle tod do

Algorithm

1. Edge cases handle karo (empty list, single node, k = 0)
2. List ka **length** nikalo aur **tail** tak jao
3. Tail ko head se connect karke circular list banao
4. `k = k % length`
5. `(length - k)` steps chalke **newTail** find karo
6. `newHead = newTail->next`
7. `newTail->next = NULL`
8. `newHead` return karo

Code (Optimal)

```
class ListNode {
public:
    int val;
    ListNode* next;
    ListNode(int x) {
```

```
        val = x;
        next = NULL;
    }
};

class Solution {
public:
    ListNode* rotateRight(ListNode* head, int k) {
        if (!head || !head->next || k == 0) return head;

        int length = 1;
        ListNode* tail = head;

        while (tail->next) {
            tail = tail->next;
            length++;
        }

        tail->next = head;
        k = k % length;

        int steps = length - k;
        ListNode* newTail = head;
        for (int i = 1; i < steps; i++) {
            newTail = newTail->next;
        }

        ListNode* newHead = newTail->next;
        newTail->next = NULL;

        return newHead;
    }
};
```

- **Time:** O(N)
- **Space:** O(1)

27. Flattening a Linked List

Brute Force Approach

Idea

Saare nodes (top level + child lists) ke values collect karo, sort karo, aur phir ek **nayi single-level linked list** bana do (sirf `child` pointer use karke).

Algorithm

1. Ek vector lo
2. `next` pointer se top-level list traverse karo
3. Har node ke liye uski `child` list traverse karke values vector me daalo
4. Vector ko sort karo
5. Sorted vector se nayi linked list banao (using `child` pointer)
6. Uska head return karo

Code (Brute Force)

```
struct Node {
    int data;
    Node* next;
    Node* child;
    Node(int x) {
        data = x;
```

```
        next = NULL;
        child = NULL;
    }
};

class Solution {
public:
    Node* buildList(vector<int>& arr) {
        Node* dummy = new Node(-1);
        Node* temp = dummy;

        for (int x : arr) {
            temp->child = new Node(x);
            temp = temp->child;
        }
        return dummy->child;
    }

    Node* flattenLinkedList(Node* head) {
        vector<int> arr;

        while (head) {
            Node* curr = head;
            while (curr) {
                arr.push_back(curr->data);
                curr = curr->child;
            }
            head = head->next;
        }

        sort(arr.begin(), arr.end());
        return buildList(arr);
    }
};
```

Complexity

- **Time:** `O(N*M log(N*M))`
- **Space:** `O(N*M)`

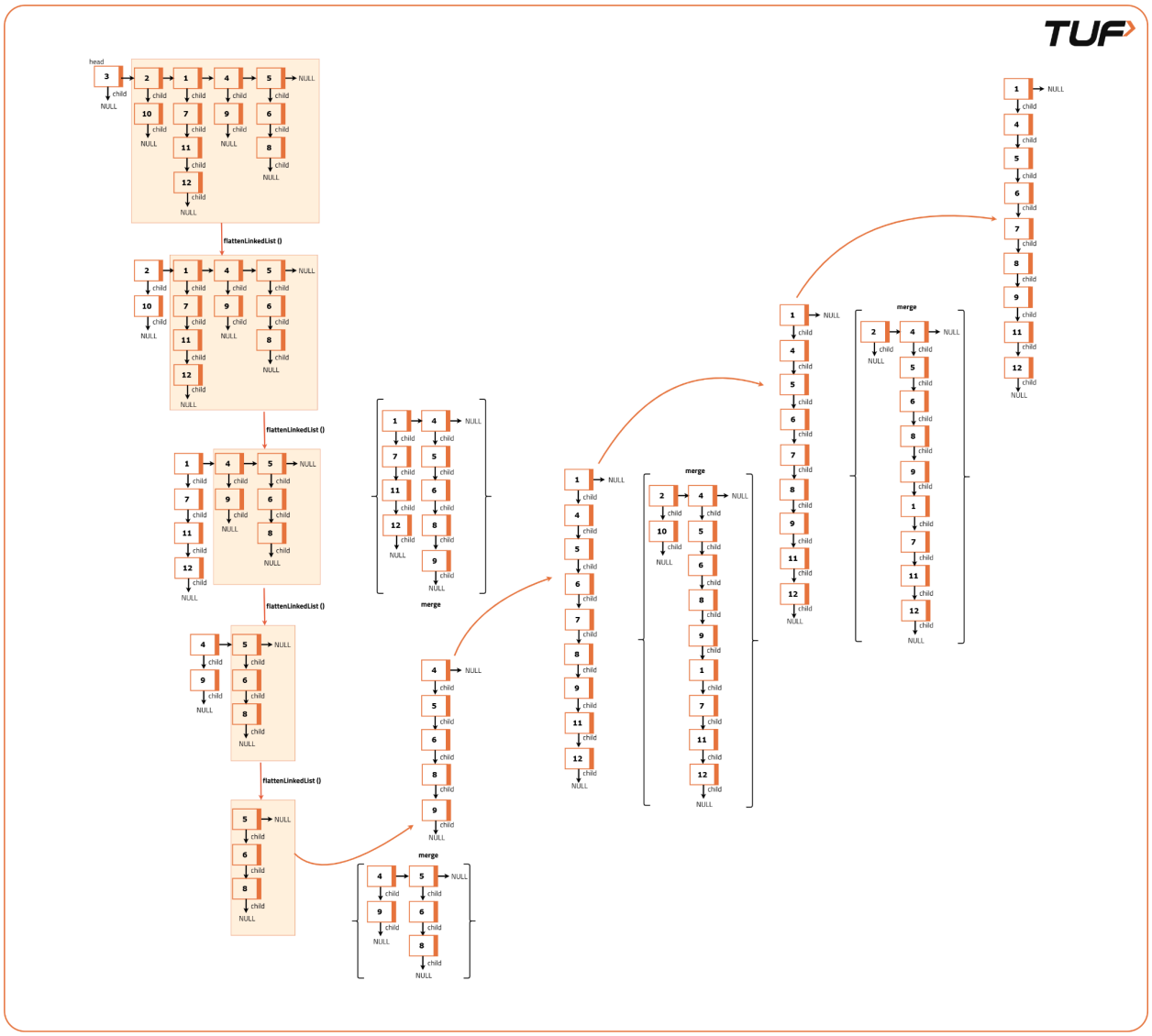
Optimal Approach

Idea

Har child list **already sorted** hai.
Hum recursion ke through **right se left** flatten karte hain aur **do sorted lists ko merge** karte jaate hain (merge sort style).

Algorithm

1. Base case: agar `head == NULL` ya `head->next == NULL` → return head
2. Recursively `head->next` ko flatten karo
3. Ab current list (`head`) aur flattened list ko merge karo
4. Merge hamesha `child` pointer se hoga
5. `next` pointer ko NULL rakho



Code (Optimal)

```
struct Node {
    int data;
    Node* next;
    Node* child;
    Node(int x) {
        data = x;
        next = NULL;
        child = NULL;
    }
```

```
};

class Solution {
public:
    Node* merge(Node* a, Node* b) {
        if (!a) return b;
        if (!b) return a;

        Node* result;
        if (a->data < b->data) {
            result = a;
            result->child = merge(a->child, b);
        } else {
            result = b;
            result->child = merge(a, b->child);
        }
        result->next = NULL;
        return result;
    }

    Node* flattenLinkedList(Node* head) {
        if (!head || !head->next) return head;

        head->next = flattenLinkedList(head->next);
        head = merge(head, head->next);

        return head;
    }
};
```

Complexity

- **Time:** `O(N*M)`
- **Space:** `O(N)` (recursion stack)

28. Clone Linked List with Random and Next Pointer

Create a 'deep copy' of the given linked list and return it.

```
Example 1:
Input: [[1, -1], [2, 0], [3, 4], [4, 1], [5, 2]]
Output: 1 2 3 4 5, true
Explanation: All the nodes in the new list have same corresponding
values as original nodes.
All the random pointers point to their corresponding nodes in the new
list.

'true' represents that the nodes and references were created new.
```

Brute Force Approach

Algorithm

1. Traverse the original list and create a copy node for every original node.
2. Store mapping between **original node → copied node** using a hashmap.
   x
3. Traverse the list again:
   - Set `next` pointer of copied node using the map.
   - Set `random` pointer of copied node using the map.
4. Return the copied head corresponding to original head.

Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Node {
public:
    int data;
    Node* next;
    Node* random;

    Node(int x) {
        data = x;
        next = NULL;
        random = NULL;
    }
};

class Solution {
public:
    Node* copyRandomList(Node* head) {
        if (!head) return NULL;

        unordered_map<Node*, Node*> mp;
        Node* temp = head;

        while (temp) {
            mp[temp] = new Node(temp->data);
            temp = temp->next;
        }

        temp = head;
        while (temp) {
            mp[temp]->next = mp[temp->next];
            mp[temp]->random = mp[temp->random];
            temp = temp->next;
        }

        return mp[head];
    }
};
```

Complexity

- **Time Complexity:** O(N)
- **Space Complexity:** O(N)

Optimal Approach

Algorithm

1. Traverse the list and insert copied nodes **in between** original nodes.
2. Traverse again to set `random` pointers of copied nodes.
3. Traverse once more to **separate original and copied lists**.
4. Return head of the copied list.

Code (Optimal)

```
#include <iostream>
using namespace std;

// Node class to represent
// elements in the linked list
class Node {
public:
    // Data stored in the node
    int data;
     // Pointer to the next node
    Node *next;
    // Pointer to a random
    // node in the list
    Node *random;

    // Constructors for Node class
    Node() : data(0), next(nullptr), random(nullptr){};
    Node(int x) : data(x), next(nullptr), random(nullptr) {}
    // Constructor with data,
    // next, and random pointers
```

```
    Node(int x, Node *nextNode, Node *randomNode) :
            data(x), next(nextNode), random(randomNode) {}
};

// Function to insert a copy of each
// node in between the original nodes
void insertCopyInBetween(Node* head){
    Node* temp = head;
    while(temp != NULL){
        Node* nextElement = temp->next;
        // Create a new node with the same data
        Node* copy = new Node(temp->data);

        // Point the copy's next to
        // the original node's next
        copy->next = nextElement;

        // Point the original
        // node's next to the copy
        temp->next = copy;

        // Move to the next original node
        temp = nextElement;
    }
}

// Function to connect random
// pointers of the copied nodes
void connectRandomPointers(Node* head){
    Node* temp = head;
    while(temp != NULL){
        // Access the copied node
        Node* copyNode = temp->next;

        // If the original node
        // has a random pointer
        if(temp->random){
             // Point the copied node's random to the
             // corresponding copied random node
            copyNode->random = temp->random->next;
        }
        else{
             // Set the copied node's random to
             // null if the original random is null
            copyNode->random = NULL;
        }

         // Move to the next original node
        temp = temp->next->next;
```

```
    }
}

// Function to retrieve the
// deep copy of the linked list
Node* getDeepCopyList(Node* head){
    Node* temp = head;
     // Create a dummy node
    Node* dummyNode = new Node(-1);
    // Initialize a result pointer
    Node* res = dummyNode;

    while(temp != NULL){
        // Creating a new List by
        // pointing to copied nodes
        res->next = temp->next;
        res = res->next;

        // Disconnect and revert back to the
        // initial state of the original linked list
        temp->next = temp->next->next;
        temp = temp->next;
    }

     // Return the deep copy of the
     // list starting from the dummy node
    return dummyNode->next;
}

// Function to clone the linked list
Node *cloneLL(Node *head){
    // If the original list
    // is empty, return null
    if(!head) return nullptr;

     // Step 1: Insert copy of
     // nodes in between
    insertCopyInBetween(head);
    // Step 2: Connect random
    // pointers of copied nodes
    connectRandomPointers(head);
    // Step 3: Retrieve the deep
    // copy of the linked list
    return getDeepCopyList(head);
}

// Function to print the cloned linked list
void printClonedLinkedList(Node *head) {
    while (head != nullptr) {
```

```
        cout << "Data: " << head->data;
        if (head->random != nullptr) {
            cout << ", Random: " << head->random->data;
        } else {
            cout << ", Random: nullptr";
        }
        cout << endl;
         // Move to the next node
        head = head->next;
    }
}

// Main function
int main() {
    // Example linked list: 7 -> 14 -> 21 -> 28
    Node* head = new Node(7);
    head->next = new Node(14);
    head->next->next = new Node(21);
    head->next->next->next = new Node(28);

    // Assigning random pointers
    head->random = head->next->next;
    head->next->random = head;
    head->next->next->random = head->next->next->next;
    head->next->next->next->random = head->next;

    cout << "Original Linked List with Random Pointers:" << endl;
    printClonedLinkedList(head);

    // Clone the linked list
    Node* clonedList = cloneLL(head);

    cout << "\nCloned Linked List with Random Pointers:" << endl;
    printClonedLinkedList(clonedList);

    return 0;
}
```

Complexity

- **Time Complexity:** O(N)
- **Space Complexity:** O(1)